

Coin Challenge

多模态 Questioner 技术报告

系统设计、核心技术与工程改进

项目：coin_challenge

日期：2026-08-16

文档类型：系统设计与工程改进报告

项目：coin_challenge

日期：2026-08-16

摘要

本项目实现了一个用于交互式图像匹配任务的多模态 Questioner。系统在每个决策步骤中接收隐藏目标的文本描述、当前候选图片以及最近一次 Oracle 回答，并在“继续询问隐藏目标”与“判断候选是否匹配”之间选择一个动作。

当前实现以豆包/火山方舟图片理解模型作为默认多模态推理后端，在模型推理之外增加了结构化记忆、证据来源追踪、回答极性解析、确定性结论门控、输出修复和有界重试。该设计的重点不是让大模型独立控制全部流程，而是把视觉理解交给模型，把协议约束、证据一致性和失败处理交给可测试的程序控制器。

本文只说明系统使用的技术、核心设计和工程改进，不记录实验分数或评测数据。

1. 任务定义

一个 episode 包含一个固定的隐藏目标和若干按顺序出现的候选图片。Questioner 只能看到：

- 隐藏目标的文本描述；
- 当前候选图片；
- 最近一次向 Oracle 提问后得到的回答。

Questioner 每一步必须返回以下两类动作之一：

1. 提问：询问隐藏目标的一个可观察视觉属性；
2. 结论：返回 0 表示不匹配，返回 1 表示匹配。

系统要求动作始终包含 question、conclusion 和 reasoning，并保证 question 与 conclusion 恰好一个非空。候选判断正确后环境切换到下一张图片，判断错误则立即结束 episode。

任务的工程难点主要来自以下方面：

- 文本描述可能只有类别，无法直接区分同类别候选；
- Oracle 只能查看隐藏目标，Questioner 只能查看候选图；
- 同一 episode 中需要跨候选复用目标信息；
- 模型输出可能不符合 JSON 或协议；

- 错误结论的代价高于一次提问，需要平衡信息收益与询问成本；
- 远程多模态 API 存在超时、连接失败和限流风险。

2. 系统架构

系统采用“环境负责协议、模型负责感知、控制器负责证据”的分层结构。

```
eval_model.py
|
+-- 创建 Questioner 和 Oracle 客户端
+-- 选择数据 split 与 description type
+-- 执行 episode 循环并保存结果
|
v
QAEEnv (env.py)
|
+-- 提供候选 RGB 图片和目标描述
+-- 校验动作、计算奖励、切换候选
+-- 将 Questioner 的问题发送给 Oracle
|
v
YourQuestioner (Questioner.py)
|
+-- 多模态模型读取候选图
+-- 结构化目标/候选记忆
+-- 证据门控与动作修复
|
v
Model Clients (utils.py)
+-- Doubao / ModelArk
+-- Gemini
+-- Local vLLM
```

2.1 模块职责

- `Questioner.py`：定义 Questioner 接口和当前活动实现 `YourQuestioner`；
- `env.py`：实现 Gymnasium 环境、奖励、终止、图片加载和 Oracle 调用；
- `utils.py`：封装远程及本地多模态模型客户端；
- `eval_model.py`：负责命令行参数、provider 选择、评测循环和结果写入；
- `Oracle.py`：定义最小 Oracle 接口；
- `tests/`：覆盖证据、记忆、动作修复、候选切换和重试逻辑。

3. 主要技术

3.1 Python 与 Gymnasium

环境继承 Gymnasium Env，通过 `reset()` 和 `step()` 表达 episode 生命周期。NumPy 用于传递 RGB 图像数组，Pillow 用于加载和编码图片。环境层独立于具体模型，因此 Questioner 和 Oracle 可以替换而不改变评分协议。

3.2 多模态模型调用

豆包通过火山方舟的 OpenAI 兼容接口接入。客户端把文本 prompt 与一张图片编码为多模态消息，并以流式方式读取模型响应。项目同时保留 Gemini 和本地 vLLM 客户端，provider 由命令行参数和环境变量选择。

统一客户端接口为：

```
ask(*, prompt: str, images: list) -> str
```

当前实现限制每次多模态请求只包含一张图片，这与 Questioner 只能查看候选图、Oracle 只能查看目标图的权限边界一致。

3.3 紧凑 JSON 动作协议

模型被要求只返回一个紧凑 JSON 对象。提问动作示例：

```
{"action": "question", "attribute": "finish_color", "question": "...", "fallback": 0, "reasoning": "..."} 
```

结论动作示例：

```
{"action": "conclusion", "conclusion": 0, "matches": ["finish_color"], "conflicts": [], "reasoning": "..."} 
```

解析器使用 `json.JSONDecoder.raw_decode()` 搜索第一个包含有效动作的 JSON 对象，避免使用贪婪正则或脆弱字符串切片。解析层同时兼容早期嵌套 schema，以便读取旧模型输出。

3.4 图像指纹

Questioner 使用 BLAKE2b 对候选图像的 shape、dtype 和像素内容生成指纹。指纹变化表示环境已经切换候选，此时只重置候选级状态，目标级记忆继续保留。

3.5 有界重试

Questioner 和 Oracle 只对连接中断、连接拒绝和超时等临时错误执行有限重试。参数错误、额度错误和其他非临时异常不会被无限重试。该策略避免短暂网络波动直接破坏 episode，同时防止错误配置导致长时间阻塞或重复计费。

4. Questioner 核心设计

4.1 两级状态

状态分为目标级和候选级。

目标级状态在整个 episode 内保留：

- 原始目标描述；
- Oracle 问答证据；
- 标准化目标事实；
- 已询问问题和属性；
- 已回答属性集合。

候选级状态在图片切换时清空：

- 当前候选画像；
- 当前候选与目标的比较；
- 当前候选提问计数；
- 当前候选图像指纹。

这种分层防止上一候选的视觉特征污染下一候选，同时允许复用已经从 Oracle 学到的隐藏目标信息。

4.2 结构化证据账本

每次 Oracle 回答都会记录以下字段：

```
question
answer
attribute
polarity: yes | no | uncertain
candidate_fingerprint
has_target_detail
```

candidate_fingerprint 用于判断该证据是否直接针对当前候选。**has_target_detail** 表示回答是否不仅给出 Yes/No，还包含可用于后续候选比较的目标细节。

回答极性通过保守规则识别：明确以 Yes/No 同义词开头的回答分别标记为 **yes** 和 **no**；包含 unknown、unclear、not visible 等表达时标记为 **uncertain**。无法可靠识别时也归入 **uncertain**，避免把模糊回答当作支持证据。

4.3 确定性证据规则

模型提出的问题必须描述当前候选中真实可见的一个原子命题，并询问隐藏目标是否也满足该命题。因此：

- 当前候选证据出现一个可靠 **no**，表示目标与候选至少存在一个视觉冲突，控制器可直接判 **0**；
- 在仅有类别描述时，两个不同的高区分度属性均得到 **yes** 且没有冲突，控制器可直接判 **1**；
- **uncertain** 不计入匹配或冲突。

这些规则由程序执行，不依赖模型是否在 **matches** 或 **conflicts** 中使用完全相同的自然语言。

4.4 Grounded Positive Gate

类别描述下的正向结论必须满足证据落地要求：

- 不得存在当前候选直接冲突；
- 不得存在模型仍未解决的冲突；
- 至少有两个独立、非类别、非通用安装属性的匹配；
- 匹配属性必须能够关联到 Oracle 证据或目标事实。

仅仅“某属性曾被问过”不代表该属性匹配。该门控用于阻止模型把回答过的属性错误地当作正向支持。

4.5 Negative Gate

对于类别相同、描述较弱的候选，如果模型没有给出具体冲突且仍有提问预算，控制器会拒绝过早的负向结论，并要求模型提出新的高区分度问题。类别明确不一致或存在可靠冲突时，负向结论可以直接通过。

4.6 问题策略

问题必须满足：

- 只询问隐藏目标；
- 是单一 Yes/No 命题；
- 不询问图片伪影或不可见信息；
- 不重复已经询问的问题或属性；
- 优先选择颜色、部件数量、结构、边框、控制布局、配件和周围物体等区分度较高的属性；
- 避免类别、对象类型等已经由描述提供或区分度较低的属性。

默认每个候选最多提问两次。提问上限是候选级限制，切换候选后重新计数。

4.7 输出修复与回退

以下输出会被控制器拒绝：

- 无法解析的 JSON；
- 缺少属性键的问题；
- 重复问题或重复属性；
- 超过候选问题上限的问题；
- 缺少证据的正向结论；
- 同类别且无冲突时过早的负向结论。

控制器会把拒绝原因和上一响应摘要加入修复 prompt，并额外调用模型一次。问题预算耗尽时要求模型返回结论；问题预算仍存在且结论缺乏依据时要求模型提出新问题。修复仍失败时执行受正负证据门控的保守回退。

5. 上下文与记忆管理

记忆以紧凑 JSON 放入 prompt，包括目标描述、目标事实、Oracle 证据、当前候选画像、比较状态和已询问属性。

默认限制为：

- 每个候选最多 2 个问题；
- 最多保留 12 条证据；
- 结构化记忆最多 6000 字符。

当记忆超过预算时，系统按优先级裁剪旧问题、旧原始证据、候选比较细节和较旧事实。目标是保留至少少量高价值证据，同时避免本地 4096 token 模型因上下文过长而失败。

记忆使用结构化字段而非直接拼接完整对话，主要优点是：

- 可以明确区分目标事实和候选事实；
- 可以追踪证据来源；
- 可以单独裁剪低优先级字段；
- 可以由控制器执行确定性检查；
- 降低模型重复理解长对话的负担。

6. Provider 与配置设计

`utils.py` 提供统一的远程模型抽象，并通过 `create_doubao_client()` 创建火山方舟客户端。配置从仓库 `.env` 读取，不存在时兼容 `$HOME/.env.ml`。

豆包相关配置包括：

- API Key 与图片理解端点；
- API base URL；
- 请求超时；
- 输出长度；
- thinking 开关；
- Questioner 和 Oracle provider；
- Questioner 独立模型 ID。

API 客户端设置有限超时并关闭 SDK 自动重试，由上层根据错误类型决定是否重试，从而避免 SDK 与业务层叠加重试。

7. 环境与评测工程

7.1 动作校验

环境检查动作是否包含 `question` 和 `conclusion`，并使用异或条件保证两者恰好一个非空。结论最终转换为布尔匹配判断，但 Questioner 接口仍明确要求返回整数 0 或 1。

7.2 图片安全加载

加载图片前读取文件头，检测 Git LFS pointer。发现指针时提供明确恢复命令，而不是让 Pillow 抛出难以理解的格式异常。

7.3 数据 split

评测器支持 `train`、`val` 和 `test`，并支持六种 `description type`。拆分脚本使用固定随机种子生成可复现的训练、验证和测试文件，同时保留原始训练集。

7.4 结果记录

每种 `description type` 生成独立 gzip JSON，保存动作、回答、reasoning、候选图、结论数、提问数、奖励、完整成功状态和耗时。控制台同时打印聚合摘要。

8. 主要工程改进

8.1 从模板到可运行 Questioner

初始模板只定义接口和待实现位置。当前 `YourQuestioner` 已形成完整闭环：模型调用、prompt 构建、JSON 解析、状态更新、动作校验、修复和回退均可直接运行。

8.2 从原始问答到结构化证据

原始文本记忆只能让模型重新解释历史，无法由程序验证。当前实现增加属性、极性和候选来源，使关键结论能够被控制器审计。

8.3 从单次模型决定到混合控制

模型负责视觉描述和候选特征选择，控制器负责协议、证据门槛和确定性规则。这种混合方式降低模型格式波动和推理幻觉对环境动作的直接影响。

8.4 Provider 显式化

Questioner 和 Oracle 可以独立选择豆包、Gemini 或本地 vLLM。默认流程统一为豆包，同时保留兼容开关，减少评测脚本中手工修改客户端代码的需求。

8.5 失败边界明确化

网络临时错误使用有限重试；额度、参数和认证错误快速暴露；无效动作执行一次修复；图片指针给出恢复信息。每类失败都有明确边界，避免无限等待。

8.6 可测试性

模型客户端可以通过依赖注入替换为 `FakeClient`。离线测试覆盖记忆跨候选保留、问题上限、重复问题、属性别名、正负证据门控、输出修复、上下文预算和重试分类。

9. 安全与运维

- API Key 只从环境变量或 `.env` 加载；
- `.env` 不进入版本控制；
- 日志不打印密钥值；
- 结果、日志、虚拟环境和下载缓存视为本地产物；

- 长时间评测应在启动时使用 `nohup` 并重定向标准输出；
- 付费评测前先运行小切片；
- Questioner 与 Oracle 同时使用远程模型时，需要考虑双侧调用成本和端点限流。

10. 已知限制

1. 评测器只在一种 description type 完成后写盘，不支持逐 episode checkpoint；
2. 回答极性解析主要面向以英文 Yes/No 开头的 Oracle 输出；
3. 每个候选固定两问，没有根据描述丰富度动态调整预算；
4. 跨候选匹配仍需要模型将目标自然语言事实与当前图片对齐；
5. 图像指纹需要读取全部像素，超大图片下存在额外 CPU 开销；
6. 当前本地客户端固定连接 `localhost:8000/v1`；
7. 旧评测脚本仍包含已经过时的参数示例，不应作为权威入口。

11. 后续改进方向

11.1 自适应问题预算

根据描述信息量、已有目标事实和候选冲突强度动态选择 0、1、2 或更多问题。详细描述优先直接比较，弱描述为高价值候选保留更多询问机会。

11.2 信息增益排序

为候选可见属性估计稀有度、可观察性和预期区分能力，程序化选择最有价值的问题，而不是只依赖 prompt 中的偏好列表。

11.3 更强的目标事实抽取

把 Oracle 自然语言回答转换为标准化属性值、置信度和否定范围，使跨候选比较可以更多地由程序完成。

11.4 逐 episode checkpoint

每完成一个 episode 即原子写入结果，并在启动时加载已有 ID，实现安全断点续跑和分片合并。

11.5 可观测性

增加假阳性/假阴性、动作拒绝原因、修复次数、回答极性和属性使用频率等诊断字段，帮助定位策略损失。

11.6 强约束输出

当 provider 支持 JSON Schema 或结构化输出时，直接在 API 层约束动作格式，减少修复调用和解析分支。

12. 结论

当前系统把多模态模型能力与确定性工程控制结合起来：模型负责理解图片和生成高价值问题，程序负责证据来源、状态边界、协议合法性和失败恢复。结构化证据和有界上下文使 Questioner 能够在多个候选之间积累目标知识，同时保持行为可解释、可测试和可替换。

该架构适合继续扩展到不同远程模型、本地 VLM 和更复杂的主动视觉问答策略，并为后续的自适应提问、标准化事实抽取和断点评测提供了清晰基础。